Evidence-Based Retention: Reducing Packet Captures to a Byte Budget with Signature-Free Selection and Pre-Deletion Verification

Ishaan Suresh Mullya, Shriya P Reddy, Om KB
Information Security Lab (ICT3141), B.Tech. CCE, 5th Semester
Faculty guide: Dr. Adesh ND

***Abstract*—Full packet capture is the most complete evidence a network can produce, but its storage cost forces most organizations to delete it within days. Existing reduction methods either treat all traffic alike or keep only what detection rules already recognize. We present Evidence-Based Retention (EBR), a streaming method that reduces a capture to an exact byte budget using no signatures, labels or training data. EBR scores each flow in bits of surprise under the capture's own baseline, applies a Good–Turing correction and two-level diminishing returns, and selects one fidelity level per flow as a multiple-choice knapsack. It also charges a complete flow ledger to the same budget. We then add verify before delete, a loop that scans the original and the reduced copy with an intrusion detection system and repairs losses inside the fixed budget before the original is removed. We evaluate with Suricata and 52,778 public rules as an independent judge across three sealed tests on 24.5 GB of public captures. In a pre-registered test on scan-heavy traffic, EBR with repair preserved 99.6% of attack signatures at a 5% cap, against 84.9% for IDS-triggered capture with the same rule knowledge. Only one of five pre-registered hypotheses was supported in full. We report the failures, including a scan cap that did not generalize and a login-spray blind spot in diminishing returns.**

***Index Terms*—network forensics, packet capture, selective retention, intrusion detection, knapsack problem, pre-registration**

I. Introduction

Packet captures let an investigator see what was actually said on a network, long after alerts have fired. The cost is storage: a fully loaded 1 Gbit/s link produces 10.8 TB per day. Lossless compression does not close the gap. On our test sets gzip reached 39.9–56.5% of the original size overall, and as much as 95.6% on a single capture, since encrypted traffic is close to incompressible. Reaching 1–10% of the original size requires discarding data on purpose, so the design question is *which bytes an investigator would most regret losing*.

The established answers each lose something. Connection cutoffs, as in Time Machine [1], [2], treat every connection alike and lose late commands. Alert-triggered capture [3], [4] keeps only what a rule already recognizes, so it cannot serve rules written after the fact. Recent traffic-aware sampling such as XNET [5] favors rare destinations but operates on headers. All of these methods also share a blind spot: the decision is made once, and its cost is unknown until an investigation fails.

This paper makes four contributions:

1) EBR, a streaming, signature-free reduction method that combines surprisal with Good–Turing correction, two-level diminishing returns, per-flow fidelity selection as a multiple-choice knapsack, and a budget-charged flow ledger.
2) Verify before delete, a repair loop that measures a reduction's losses while the original still exists and corrects them inside the same byte budget.
3) A sealed, pre-registered evaluation using an independent detector, fair comparators and paired intervals over cases.
4) Negative results that bound these claims, including an ablation showing which components matter and a blind spot in the method's most effective component.

II. Related Work

Connection-prefix retention. Kornexl *et al.* [1] showed that keeping only the start of each connection preserves most value at a fraction of the volume, and Maier *et al.* [2] let a network IDS adjust storage classes. TrimPCAP [6] applies per-session trimming to existing files. Selective capping [7] removes payloads that are hard to interpret later. These methods use a fixed policy per class, whereas EBR varies depth per flow by estimated value.

Detection-driven retention. Suricata's conditional logging [4] and IDS-triggered capture [3] retain traffic that matches rules. They are precise, but retention then depends on what is known at capture time.

Traffic-aware sampling. XNET [5] samples dynamically at very high rates, favoring rarely contacted destinations and DNS using header information. It runs live, which EBR does not. EBR instead reads protocol content, calibrates itself on each capture without external feeds, and optimizes against a byte budget.

Evaluation practice. Sommer and Paxson [8] describe why anomaly-based intrusion detection is hard to deploy and to evaluate. Engelen *et al.* [9] document labeling and construction flaws in widely used intrusion datasets. We respond by using an independent detector, pre-registration [10], and data from several sources.

Novelty. In a bounded literature search we found no published method with EBR's combination of parts, or one that verifies a reduction against an IDS and repairs it inside the same budget before deletion. Retention by value is not

TABLE I
FIDELITY TIERS AND VALUE WEIGHTS

Tier	Contents	Weight w_t
0 Skeleton	SYN, FIN, RST, first and last packet	0.20
1 Head	First 1 KiB in each direction	0.50
2 Deep head	First 16 KiB in each direction	0.15
3 Turn heads	First 1 KiB of each later message turn	0.10
4 Bulk	All remaining packets	0.05

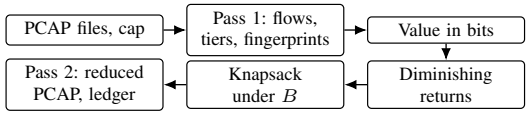


Fig. 1. The EBR pipeline. No rules, labels or training data are used.

itself new, and related patents exist. Our claim concerns the combination and its evaluation.

III. Problem Formulation

Let a capture contain flows F. Each flow f has packets partitioned into five fidelity tiers (Table I), with byte costs $c_f(t)$ for $t \in \{0, \dots, 4\}$. Choosing level ℓ_f retains all tiers up to ℓ_f, and level -1 retains no packets. Given a budget B in serialized bytes, a ledger of size $|L|$ and the PCAP framing overhead h, we seek

$$\max_{\{\ell_f\}} \sum_{f \in F} v_f(\ell_f) \quad \text{s.t.} \quad \sum_{f \in F} \sum_{t \le \ell_f} c_f(t) + |L| + h \le B. \quad (1)$$

This is a multiple-choice knapsack problem [11]. Its quality depends on the value model v_f, which must be computed without labels.

IV. Evidence-Based Retention

EBR makes two passes (Fig. 1). The first streams the capture into flows, tiers and protocol fingerprints, holding no packet content. The second copies chosen records byte for byte, so the output is a standard PCAP.

A. Fidelity Tiers

Tier positions come from TCP sequence numbers, so retransmissions fall in the tier of the data they repeat. Pure ACKs follow the data they acknowledge, which keeps retained streams usable by a reassembling IDS. Each change of speaker starts a new *turn*, and Tier 3 keeps the opening of each turn, which a plain prefix cannot do. UDP tiers follow packet counts.

B. Rarity

For a trait with value x seen n_x times among N flows, plain frequency overrates one-off values. Following Good [12], let the novelty g be the share of flows whose value occurs exactly once, and estimate

$$\hat{P}(x) = \max\left(\frac{n_x}{N},\ \frac{g}{n_x}\right), \quad s(x) = \min\big(16,\ -\log_2 \hat{P}(x)\big). \quad (2)$$

A one-off therefore scores highly only when one-offs are rare. Five traits are measured: service, flow shape, client–server pair, client fingerprint, and content skeleton. The fingerprint derives from the HTTP header layout and User-Agent, a hash of the TLS ClientHello, or the SSH banner. The skeleton replaces the words of a URL with a placeholder and keeps its punctuation, so that `/page?id=1'%20OR%201=1--` differs from `/page?id=7` without any rule describing SQL injection. With $e_{(i)}$ the excesses over each trait's median in descending order, rarity is

$$R(f) = e_{(1)} + 0.25 \sum_{i \ge 2} e_{(i)}. \quad (3)$$

C. Automation and Pivoting

Rarity fails on floods, because a flood's traits become common. For a group of $n \ge 8$ flows sharing client, server, port and protocol,

$$A(f) = \log_2(n) \cdot \max\big(1 - \sigma_{\text{size}},\ 1 - \sigma_{\text{gap}},\ \phi\big), \quad (4)$$

where σ denotes a coefficient of variation clipped to 1 and ϕ is the share of failed connections. A client whose failures reach $d \ge 8$ distinct endpoints scores at least $\log_2 d$.

Let $b(f) = \max(R, A)$. A host's mass is $m(h) = \sum 2^{\min(b(f)-6,\,12)}$ over the flows it initiated with $b(f) \ge 6$, and its suspicion $H(h)$ is $\log_2(1 + m(h))$ less the median across hosts, floored at zero. Each flow then receives an uplift $U(f) = 0.5 \max(H_{\text{client}}, H_{\text{server}})$. A host that was attacked also lifts its next connections to rare peers for 30 minutes, with weight 0.25.

D. Diminishing Returns

The ten-thousandth identical failed login adds little once exemplars and an exact count exist. Within a cluster, flows are ranked, and the first 16 keep full value so that rate-based rules have an unbroken burst to fire on. The rest are visited in bit-reversed order, so any prefix covers the cluster's whole time span, and each receives a discount r^{-p}, where r is its rank less 15. With D_p^C this discount in cluster C, the multiplier is

$$M(f) = D_{0.5}^{\text{exact}} \cdot \min\big(D_{0.25}^{\text{client}},\ D_{0.25}^{\text{server}},\ D_{0.5}^{\text{silent}}\big). \quad (5)$$

Exact repeats (same client, server, service, content and shape) fall steeply. Variations fall gently, because a new payload or endpoint is a new indicator. The client cluster omits the port, so a thousand-port scan counts as one behavior. A flow's value is then

$$V(f) = \big(1 + b(f) + U(f)\big) \cdot M(f). \quad (6)$$

E. Allocation

A level earns $v_f(\ell) = V(f) \sum_{t \le \ell} w_t$ over the tiers the flow actually occupies, so a bare SYN cannot outbid a real request by being cheap. For opaque flows, tiers above the head are worth a tenth. Algorithm 1 reduces each flow's levels to their upper concave hull and takes segments in order of value per byte, the standard greedy solution, which is optimal for the

Algorithm 1 Budgeted Level Selection

$S \leftarrow$ concave-hull segments of all flows, by value per byte
used $\leftarrow 0$; room $\leftarrow 0.3B$; deferred $\leftarrow [\,]$
for each segment $(f, \ell, c) \in S$ **do**
  if f is blocked **then continue**
  end if
  if f is payload-free **then**
    if $c >$ room **then** append to deferred; **continue**
    end if
    room $\leftarrow$ room $- c$
  end if
  if used $+ c \le B$ **then** used $\leftarrow$ used $+ c$; $\ell_f \leftarrow \ell$
  else block f
  end if
end for
for each deferred segment (f, ℓ, c) with used $+ c \le B$ **do**
  used $\leftarrow$ used $+ c$; $\ell_f \leftarrow \ell$
end for

fractional relaxation of (1). In the algorithm, B is the budget net of the ledger and framing.

Payload-free flows are cheap enough that a scan storm can outbid content on price. On one capture they took 186 MB of a 227 MB packet budget. They are therefore limited to a 30% share on the first pass, and they may use whatever content leaves unused. This second constraint makes the result a heuristic rather than the fractional optimum.

F. Flow Ledger

Every flow is recorded in a gzip-compressed table with its endpoints, times, packet and byte counts, application and state. When space is tight, low-value flows fold into grouped rows that keep exact totals. The ledger is limited to 10% of the budget and is charged against B, so no comparator is disadvantaged.

V. Verify Before Delete

Retention is usually a blind bet, yet the complete capture exists until the moment it is deleted. Algorithm 2 scans both copies, finds the flows behind each lost detection, and selects again with those flows outbidding all others. The cap holds because the lowest-value evidence makes room. An alert's first repair restores its flows by five-tuple. If the alert is still lost, the next round also adds up to 64 neighboring flows between the same hosts in its time window, because rate-based rules need several flows close together.

The initial selection still uses no rules, so protection against unknown threats still rests on EBR. The IDS runs once per batch and repairs only what it can see.

VI. Evaluation Method

A. Detector and Measures

Suricata 7.0.3 with 52,778 ET Open rules [4], [13] is the judge. EBR never sees these rules during selection. We compare alerts from each reduced copy with those from the full capture. Alerts are restricted to attack categories. The primary measure is *attack signature recall*, the share of distinct rules that still fire. *Alert instance recall* counts distinct combinations of rule and host pair. *Later-rule recall* covers rules published on or after the capture date. A case with no attack alerts on the full capture is not scored.

Algorithm 2 Verify Before Delete

$Q \leftarrow$ IDS alerts on the full capture from admissible rules
forced $\leftarrow \emptyset$; mask $\leftarrow$ EBR(B)
for round $= 1, 2$ **do**
  lost $\leftarrow Q \setminus$ IDS alerts on the reduced copy
  if lost $= \emptyset$ **then break**
  end if
  add the flows behind lost alerts to forced
  for alerts repaired before and still lost, add up to 64 time-neighbors
  $V(f) \leftarrow 1000 \cdot \max V$ for $f \in$ forced
  mask $\leftarrow$ select again under B
end for
return mask, then delete the original

TABLE II
Sealed Test Sets

Test	Size	Packets	Scored	Sources
First	11.28 GB	35.0 M	6/6	CIC (5 days), WRCCDC 2018
Second	7.03 GB	15.1 M	5/6	WRCCDC, CTU-Mixed, CIC
Third	6.15 GB	34.0 M	8/9	IoT-23, WRCCDC 2017/18

B. Data

Table II lists the three sealed sets. They draw on CSE-CIC-IDS2018 [14], WRCCDC competition captures with live human red teams [15], CTU-Mixed captures from the group behind the CTU-13 data [16], and IoT-23 [17]. Separate development cases were used for tuning.

C. Comparators

All methods receive the same serialized-byte budget at caps of 1, 2, 5 and 10%. Random packet and random flow selection use three seeds. The flow cutoff, after Time Machine, is fitted to every budget, which makes it stronger than a fixed setting. Rare destinations is inspired by XNET but is not a reproduction, since it is offline and uses no popularity feeds. IDS-triggered + fill keeps alerted flows and spends what remains on a fitted cutoff. It receives real alerts from rules older than each capture, an advantage EBR's first selection never has. Repair in the third test used the same restricted rules.

D. Sealing and Pre-Registration

Before each test, code, settings and scoring were hashed with SHA-256 and recorded, and only then were the captures opened. For the third test, the plan itself was included among 22 hashed files. It fixed the primary measure, the pass rule and these hypotheses:

- H1: on scan-heavy cases, EBR with the scan cap beats the first version at 5% and 10%.
- H2: repair beats EBR alone at every cap.
- H3: repair beats IDS-triggered + fill at every cap.
- H4: repair costs later-rule recall at most 5 points on average at any cap.
- H5: a hidden-family estimate of unknown-threat recall beats guessing the mean, given at least 10 usable pairs.

TABLE III
ATTACK SIGNATURE RECALL (%) AT A 5% CAP

Test	EBR	Repair	IDS+fill	Cutoff	Rare
First	91.7	–	86.5	62.8	69.2
Second	94.4	–	85.9	77.5	79.9
Third	84.7	**99.6**	84.9	34.4	14.1

A blind screen using packet headers alone, run before any detection, marked 7 of the 9 captures as scan-heavy. The first frozen run stopped on a PCAPNG file before any scoring. The file was converted, Scapy confirmed 18,890,171 packets with zero mismatches, and no frozen file changed.

E. Statistics

A hypothesis counts as supported at a cap only if the mean paired difference favors it and the 95% bootstrap interval [18], from 10,000 resamples of cases, excludes zero.

VII. RESULTS

A. Across the Three Tests

Table III gives signature recall at a 5% cap. In the first test, EBR beat the tuned cutoff and rare destinations with intervals excluding zero at every cap, and it could not be separated from IDS-triggered capture. It preserved all 8 labeled attacks that the detector finds on complete captures at 5% and 10%. Its ledger gave the exact attacker flow count in all five labeled cases at 5%. The weak spot was a human red-team capture (60.7% against 78.6%), where diagnosis found payload-free flows taking 82% of the budget, which led to the scan cap.

In the second test, EBR led every comparator on signature recall at every cap, clearing the interval against IDS-triggered capture at 5% and 10% (+8.5 and +6.5 points). However, the scan cap never engaged, because payload-free flows would have taken at most 3.4% of the budget. The third test was designed to fix this gap.

B. Third Sealed Test

Fig. 2 and Table IV show the results on scan-heavy data. Table V gives the verdicts: one hypothesis was supported in full. Repair beat EBR alone at every cap (Fig. 3). It beat IDS-triggered capture clearly at 2, 5 and 10%, but not at 1%, so H3 fails under the pre-registered rule. EBR alone fell behind IDS-triggered capture at 1% and 2%, drew level at 5% and led at 10%, so its earlier lead does not hold on all traffic.

Costs of repair. Repaired flows used 15.6, 13.1, 5.0 and 1.4% of the budget at the four caps. Repair made 3 of 64 case comparisons worse, and 249 of 2,041 usable alerts stayed lost at 5%, mostly rate-based scan rules. Scanning the original took about 62 s per GB in our lab, in addition to about 20 s per GB for profiling and scoring.

Measurement noise. Scanning identical selections twice changed 12 of 32 comparisons, by 0.4 points of instance recall on average and 6.1 at most. Signature recall moved by 0.4 points on average and 12.5 at most, on a case with only 8 signatures. Single-threaded scanning did not remove this, so gaps of a point or two are not meaningful.

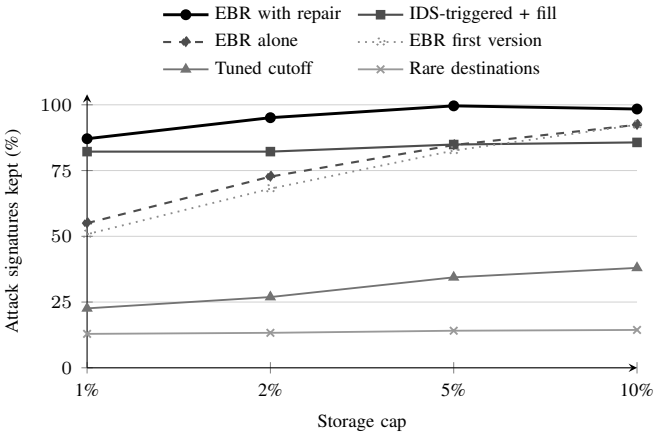


Fig. 2. Third sealed test: attack signature recall, mean over 8 scored cases.

TABLE IV
THIRD TEST: RECALL (%) BY CAP

Attack signatures	1%	2%	5%	10%
EBR with repair	**87.1**	**95.1**	**99.6**	**98.4**
EBR alone	55.0	72.7	84.7	92.4
EBR first version	50.8	68.1	82.6	92.4
IDS-triggered + fill	82.2	82.2	84.9	85.7
Tuned flow cutoff	22.6	26.9	34.4	38.0
Rare destinations	12.9	13.3	14.1	14.4
Alert instances	**1%**	**2%**	**5%**	**10%**
EBR with repair	**79.3**	**85.3**	**92.2**	**94.2**
EBR alone	40.3	49.7	74.2	85.3
IDS-triggered + fill	73.1	73.3	82.0	82.2

C. Ablation

Table VI removes one component at a time on the first test's data. Diminishing returns accounts for most of the benefit, cutting recall at 1% from 87.7% to 51.5% when removed. Removing pivoting or turn heads changed nothing measurable. Removing the ledger raised labeled attacks caught at 1% from 5 to 7, which is the price of exact flow accounting.

VIII. DISCUSSION

Why the scan cap failed. On IoT captures a botnet host produces millions of scan flows but few with content, and that content usually fits anyway. The cap helped 3 of 8 cases and did not lower signature recall in any. It is a safeguard against a rare failure, not a general gain.

Login sprays. On the Hide and Seek capture, EBR alone lost 110 of 158 attack alert instances at a 10% cap, mostly Telnet logins with default passwords, the technique Mirai made well known [19]. Attempts share size and shape and differ only in the password, so diminishing returns treats them as repeats. This is a real blind spot in the component that does the most work. Repair corrects it when a rule exists.

TABLE V
PRE-REGISTERED VERDICTS

	Verdict	Difference in points [95% interval]
H1	Not supported	5%: +2.8 [0.0, +8.3]; 10%: 0.0 [0.0, 0.0]
H2	Supported	1%: +32.1 [+10.6, +54.5]; 2%: +22.4 [+3.4, +43.3]; 5%: +14.9 [+3.4, +28.8]; 10%: +7.6 [+0.4, +19.7]
H3	Not supported	1%: +4.8 [−14.4, +21.4]; 2%: +12.9 [+3.6, +23.4]; 5%: +14.7 [+4.2, +25.7]; 10%: +12.7 [+3.3, +22.1]
H4	Not supported	1%: −1.7; 2%: −7.5 (four cases); 5%: 0.0; 10%: 0.0
H5	Not scoreable	8 of 10 pairs. Error 24.5 vs. 14.6 for guessing

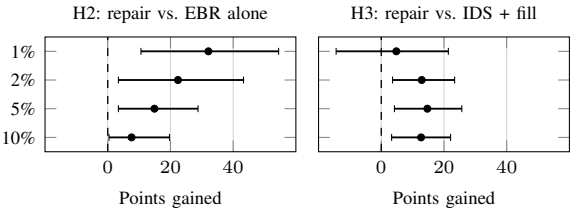


Fig. 3. Paired differences with 95% bootstrap intervals. The dashed line marks zero.

Later-published rules. In the first test, rules newer than the capture did not favor EBR as we had expected (85.0% against 92.9% at 5%), because new rules mostly fired on flows that older rules had flagged. In the third test H4 narrowly failed on only four cases. The blind-spot estimate looked promising on unsealed data, then did worse than guessing on the few usable pairs, so we regard it as unproven.

Threats to validity. Each test has few independent cases, so intervals are wide. Many cases come from one generated dataset, which has known limitations [9]. IoT-23 captures are the first 300 MB of longer recordings and are dominated by one infected host. The WRCCDC 2018 slices share an event with a development capture, at different hours. All three test sets have now been seen, so they cannot serve as sealed data again. We measured what a rule-based detector can find, not what a human analyst can reconstruct. Reduced captures can also raise a few alerts absent from the full capture, and every method showed this.

Limitations. EBR runs offline in two passes and is not a detector, since its suspect list is a triage aid. Discarded data cannot be recovered. Rate-based rules are hard to repair. At about 82 s per GB, a single machine like ours could sustain roughly 100 Mbit/s, so faster links would need parallel batches. Pre-deletion scanning requires an IDS and compute. SHA-256 freezes show that files are unchanged, which is not the same as a chain of custody [20].

IX. CONCLUSION

EBR shows that a signature-free, value-driven method can reach 1–10% of the original size while preserving most detections, and that behavior-aware diminishing returns is what makes this work under tight caps. The stronger result is verify before delete. Measuring losses while the original exists and repairing them inside the same budget kept 99.6% of attack signatures at a 5% cap in a pre-registered sealed test. Most of our pre-registered hypotheses did not pass in full, and we report that plainly.

Future work should make content comparison sensitive to credentials so that sprays stop resembling repeats, retest the blind-spot estimate on captures with more later-published rules, evaluate on operational networks, build a streaming version, and measure analyst performance on reduced evidence.

TABLE VI
ABLATION: SIGNATURE RECALL (%), FIRST TEST

Variant	1%	2%	5%	10%
Full EBR	87.7	90.5	91.7	91.1
No diminishing returns	51.5	67.3	77.8	77.8
Tiers and knapsack only	51.1	63.4	75.5	76.5
No rarity	83.8	88.6	91.1	92.3
No automation	87.1	89.9	90.5	91.7
No pivoting	87.7	90.5	91.7	91.1
No turn heads	87.7	90.5	91.7	91.1
No ledger	87.7	90.5	91.7	92.3

DATA AND CODE AVAILABILITY

All captures are from public sources. The implementation, pre-registered plan, SHA-256 freeze records and per-case results are kept in the project archive and are available from the authors.

ACKNOWLEDGMENT

The authors thank Dr. Adesh ND for guidance during this work. Under IEEE's policy on AI-generated content, the authors disclose that Claude (Anthropic), running in Claude Code, was used for the EBR implementation, the evaluation harness, all figures, and the drafting of every section of this paper, at a substantial level. An earlier version of the project was produced with ChatGPT (OpenAI). The experimental results come from executed code and saved outputs, and the authors are responsible for the content.

REFERENCES

[1] S. Kornexl, V. Paxson, H. Dreger, A. Feldmann, and R. Sommer, "Building a time machine for efficient recording and retrieval of high-volume network traffic," in *Proc. ACM SIGCOMM Internet Meas. Conf.*, 2005, pp. 267–272.
[2] G. Maier, R. Sommer, H. Dreger, A. Feldmann, V. Paxson, and F. Schneider, "Enriching network security analysis with time travel," in *Proc. ACM SIGCOMM*, 2008, pp. 183–194.
[3] Z. Gaster, "Rethinking full packet capture: Evaluating IDS-triggered PCAP," SANS Institute, White Paper, Sep. 2026.
[4] Open Information Security Foundation, "Suricata user guide." Accessed: Oct. 2026. [Online]. Available: https://docs.suricata.io
[5] T. Papastergiou *et al.*, "XNET: Intelligent dynamic sampling for high-speed network security monitoring," arXiv:2608.18349, Aug. 2026.
[6] E. Hjelmvik, "Don't delete PCAP files – trim them!" Netresec Blog, Dec. 2017.
[7] V. Uceda, M. Rodríguez, J. Ramos, J. L. García-Dorado, and J. Aracil, "Selective capping of packet payloads for network analysis and management," in *Proc. Traffic Monit. Anal. (TMA)*, ser. LNCS, vol. 9053, 2015.

[8] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 305–316.

[9] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops*, 2021, pp. 7–12.

[10] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Nat. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[11] P. Sinha and A. A. Zoltners, "The multiple-choice knapsack problem," *Oper. Res.*, vol. 27, no. 3, pp. 503–515, 1979.

[12] I. J. Good, "The population frequencies of species and the estimation of population parameters," *Biometrika*, vol. 40, nos. 3–4, pp. 237–264, 1953.

[13] Proofpoint, "Emerging Threats Open ruleset." Accessed: Oct. 2026. [Online]. Available: https://rules.emergingthreats.net

[14] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.

[15] Western Regional Collegiate Cyber Defense Competition, "Public archive." Accessed: Oct. 2026. [Online]. Available: https://archive.wrccdc.org

[16] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Comput. Secur.*, vol. 45, pp. 100–123, 2014.

[17] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Zenodo, 2020. [Online]. Available: https://zenodo.org/record/4743745

[18] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[19] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1093–1110.

[20] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to integrating forensic techniques into incident response," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Special Publication 800-86, Aug. 2006.